



Deployment Review

Final Report



Sky – PAU Deployments

September 2026

Prepared for Spark Foundation

Table of Contents

Project Summary	3
Project Scope	3
Project Overview	3
Protocol Overview	3
PAU system background	4
Components and call path	4
Parallel operation and the pre-activation state	5
Review description and scope	6
Purpose and reviewed material	6
Verification work	9
Address origin and transaction history	9
Bytecode and deployment bindings	9
Integration and selector configuration	10
Roles, Safe and existing proxy	11
Inactive state and functional checks	12
Shared-proxy architecture and approval limits	12
Findings Summary	14
Severity Matrix	14
Detailed Findings	15
Informational Severity Issues	16
I-01: An attacker could have claimed the expected Controller address during deployment	16
Conclusions	18
Disclaimer	19
About Certora	19



Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Sky - PAU Deployments	https://github.com/sparkdotfi/spark-address-registry	900e8601a1c09a1b05c522637a4a52a8f1f7cacd	Solidity

Project Overview

This document describes the security review of **Sky - PAU Deployments**. The work was undertaken from **September 24th, 2026** to **September 25th, 2026**.

The following contract list is included in our scope:

`src/Arbitrum.sol`

`src/Base.sol`

The team performed a deployment review of all the files in scope. During the review, the Certora team discovered an informational issue in the code, as listed on the following page.

Protocol Overview

Spark moves protocol liquidity through an asset-holding **ALM_PROXY** and controllers authorized to instruct it. On Arbitrum and Base chains, [PR #115](#) registers a new Parallelized Allocation Unit (PAU) Controller beside the existing **ForeignController**.

The new Controller is configured with one Circle Cross-Chain Transfer Protocol (CCTP) v2 integration, intended to send native USDC from the existing proxy to Spark's Ethereum proxy. The legacy Controller and its CCTP v1 route remain in place.



PAU system background

Components and call path

The `ALM_PROXY` holds the assets. An account with its `CONTROLLER` role can instruct the proxy to make normal calls and delegatecalls; the proxy does not restrict those calls to particular assets or destinations. The new [PAU Controller](#) limits its callable surface through a selector dispatch map: an incoming function selector must be wired to a facet, or the call reverts. A facet is a module whose code runs in the Controller's context. The [CCTP facet](#) is the only facet wired in the reviewed deployments.

The Beacon stores approved integration configurations, including the facet address and the mapping between public Controller selectors and facet selectors. The Controller does not automatically adopt a Beacon change. An administrator must call `updateIntegrations` to copy the selected configuration into the Controller. This distinction matters because the deployed Beacon state alone does not prove which functions the Controller can currently execute. We therefore checked the actual dispatch state on both contracts. The [Beacon code](#) defines the source configuration copied by the Controller.

The Controller reads permissions from `PAU_ACCESS_CONTROLS` and spending budgets from its dedicated `PAU_RATELIMITS`. Normal operation starts at `ALM_RELAYER_MULTISIG`, which is an actor on `PAU_ADMINISTERED_AGENT`. The Agent forwards the call to the Controller and is the sole holder of `ALLOCATOR_ROLE` in the new AccessControls contract. The CCTP facet's `transfer` function requires `ALLOCATOR_ROLE`, while `setDomainParameters` requires `DEFAULT_ADMIN_ROLE`. A direct relayer call lacks the allocator role, and an Agent-routed call lacks the admin role.

At the reviewed blocks, the grantor Safe could add Agent actors and the freezer Safe could remove them. Spark's chain-specific `SPARK_EXECUTOR` was the sole administrator of the Beacon, AccessControls, RateLimits, and Agent. Under the planned handover, `SPARK_BEACON DEFAULT_ADMIN_ROLE` moves to Sky's `L2GovernanceRelay` (`Arbitrum.SKY_GOV_RELAY` in the Oct 8 spell; `Base.SKY_GOV_RELAY` in its activation spell), while `SPARK_EXECUTOR` retains admin on `PAU_ACCESS_CONTROLS`, `PAU_RATELIMITS`, and the Agent. These role checks govern ordinary relayer calls; administrators can still change permissions and Controller wiring.

When active, a CCTP transfer consumes both an aggregate PAU limit and a destination-domain PAU limit. The facet then reads the configured recipient and fee bounds, approves the exact

USDC amount from the proxy to Circle's TokenMessengerV2, calls `depositForBurn` in chunks if needed, and clears the allowance. The approved recipient for Ethereum domain `0` is intended to be Spark's Ethereum `ALM_PROXY`. If a required permission, rate limit or destination setting is missing, the call reverts.

Parallel operation and the pre-activation state

The new and legacy Controllers point to the same existing `ALM_PROXY`, but each has its own rate limits. The primary relayer can use the legacy route and is the initial actor for the PAU route. Once governance activates the new route, the two CCTP budgets will add rather than share a single limit. Stopping activity on both routes likewise requires action on each Controller or its associated actor or relayer permissions. Each Controller also has its own reentrancy guard; the reviewed CCTP facet uses fixed USDC and TokenMessengerV2 targets, while a future facet would require a fresh assessment of its interactions with the shared proxy.

The registered deployment is intentionally **pre-activation**. At the reviewed blocks, the PAU Controller did not hold `CONTROLLER` on the shared proxy, Ethereum domain `0` had no PAU recipient, and both PAU CCTP limits were zero. The legacy Controller retained its proxy authority.

The activation plan would grant proxy authority to the PAU Controller, set the Ethereum recipient and fee caps, assign PAU limits, retain the relayer Safe and add a Spark hot wallet as the Agent's two actors, add a Soter revoker, replace the grantor Safe with a Soter grantor, and grant PAS admin on both `PAU_ACCESS_CONTROLS` and `PAU_RATELIMITS`. The PAS grants may occur in the Oct 8 or following spell, depending on Sky's PAS configuration. We have not verified an executed spell or its live-route parameters. The observed deployment is approved as pre-activation state; the CCTP-only architecture is conditionally supported, while activation approval awaits that separate review.



Review description and scope

Purpose and reviewed material

This review asked whether the addresses added by PR #115 (registry commit [900e8601a1c09a1b05c522637a4a52a8f1f7cacd](#)) identify the intended production contracts **on the named chain**, and whether their deployed code, bindings, CCTP wiring and authority match the stated pre-activation configuration.

This is a deployment verification: the registry diff defines the addresses to check; a script's predicted output or an explorer label by itself is insufficient to establish what was actually deployed. We compared independent forms of evidence: the pinned source and build settings, the deployment inputs and records, transaction calldata and receipts, factory events and CREATE addresses, deployed runtime code, historical role events, and direct contract reads at fixed blocks.

The registry adds eight new contracts and one existing grantor Safe on each chain. The same grantor Safe address is used on both networks, but the networks hold distinct contracts and histories. In particular, [0x86036CE5d2f792367C0AA43164e688d13c5A60A8](#) is the new Beacon on Arbitrum and the **legacy Controller** on Base. Every address below was checked with its chain identity, rather than by address string alone.

Registry component	Arbitrum One	Base
SPARK_BEACON	0x86036CE	0x7ac9618
SPARK_PAU_FACTORY	0x3968a022	0x011A115
SPARK_ADMINISTERED_AGENT_FACTORY	0xCBAOC0a	0xD711Dbf
CCTP_FACET	0xeCCA0D2	0xb22d50c
PAU_ACCESS_CONTROLS	0x8386f81	0xE593c8c



PAU_ADMINISTERED_AGENT	0x0745aae	0x70E46bA
PAU_CONTROLLER	0x04ACB9e	0xd864bF1
PAU_RATELIMITS	0x4824C43	0x5E311e8
PAU_GRANTOR_MULTISIG	0x4B61AOE	0x4B61AOE

The source versions used for bytecode comparison were `diamond-pau` v1.14.0 at `cbf71b2ac840ca9288eb867d3dd354e08089e1d7` and `pau-administered-agent` v1.0.0 at `bfaaf709a8664d74d12604455f0365a0a12439cf`. We used `spark-pau-deploy` at `8982d6441940d1c04a9d68e3e1979df88c97cc0f` to interpret the intended inputs and call sequence. The build used Solidity 0.8.34, the optimizer with 200 runs, and the Cancun EVM target. The scripts and tests were reference material for the intended deployment; the chain results were the basis for the conclusions.

The deployment review checked:

- Contract identity and runtime bytecode for all eight new contracts on each chain, to establish code provenance.
- Constructor values, immutables, and stored Controller bindings, to establish which Beacon, proxy, USDC, and Circle contracts the code actually uses.
- The complete CCTP selector maps on the Beacon and Controller, including the match to the Sky Ethereum Beacon and checks for unexpected or colliding routes.
- The new contracts' complete role membership and removal of the deployer's permissions, to rule out retained or extra authority.
- The existing proxy's code and PR-relevant roles before and after deployment, to detect an unintended custody change.
- Actual transaction provenance, factory CREATE nonces, events, and the pre-activation state, to test both the intended deployment and factory front-running concern.
- The relayer, grantor, and freezer Safes' observed owners and thresholds, because those Safes hold live Agent or legacy-controller permissions. We did not verify their organizational signer approval or the grantor Safe's implementation version, modules, guard, or fallback handler.
- The deployment/configuration scripts and their tests, including gaps in script checks and test assertions, and a first-principles review of the current parallel-controller architecture.



This report does not re-audit the PAU or Agent source code, the legacy Controller and proxy as standalone contracts, or Circle's contracts. It does not verify the future governance spell, its final limits and recipient, destination-chain minting, or any later facet addition.

The shared-proxy assessment below covers the current CCTP-only integration and the proposed activation assumptions; it is not a completed sign-off on the final activated architecture. We checked the state before deployment, after contract creation, after configuration at Arbitrum block [506469723](#) and Base block [51606874](#), and at later snapshots. The latest state conclusions are anchored to Arbitrum block [507147653](#) on 20 September 2026 and Base block [51684549](#) on 23 September 2026. These checkpoints distinguish deployment changes from later activity; changes after the latest blocks require a new state check.



Verification work

Address origin and transaction history

On each chain, we compared the eight new registry addresses with the deployment record and the contracts created by actual transactions. Four contracts were created directly by the deployer: Beacon, PAUFactory, AdministeredAgentFactory, and CCTPFacet. PAUFactory then created AccessControls, RateLimits, and Controller at its first three **CREATE** nonces.

AdministeredAgentFactory created the Agent at its first **CREATE** nonce. We independently calculated those addresses and matched them to receipts and factory events. This ties each registered child to observed chain history instead of a predicted output file.

The deployer submitted 23 successful PR-listed transactions with contiguous EOA nonces on each network: Arbitrum nonces 12–34 and Base nonces 20–42. Eight transactions deployed contracts and 15 configured them. We decoded every listed call's sender, target, arguments, receipt status, and order, including all ten CCTP selector pairs and administration handoff calls. All 24 expected receipt events per chain matched. None of those transactions targeted or emitted a log from the existing **ALM_PROXY**. We also scanned the factory histories and their child role events beyond the listed transactions; no successful unexpected child creation or administrator was found.

Arbitrum's four child calls succeeded in blocks **506440776**, **506440780**, **506440784**, and **506440788**, with no extra successful factory creation in the intervening deployment window. On Base, PAUFactory creation, AgentFactory creation, CCTPFacet creation, and the four expected child calls occupied consecutive transaction indices 359–365 in block **51606638**. The PAUFactory nonce was exactly 4 and the AgentFactory nonce exactly 2 after configuration and at the later checks on both chains. Source code and expected addresses alone could not rule out another caller using a public factory first; the transaction, nonce, constructor, and role evidence rules out a successful factory–nonce front–run in these deployments. A reverted or dropped attempt would not create a child and cannot be established from these state checks.

Bytecode and deployment bindings

We compiled the verified Standard JSON input using the pinned source and compared all 16 deployed runtimes with direct chain reads. The ten Beacon, AdministeredAgentFactory,



AccessControls, RateLimits, and Agent runtimes match byte for byte, including metadata. The six PAUFactory, Controller, and CCTPFacet runtimes match after masking only compiler-declared immutable positions; the decoded immutable values are the intended Beacons, chain-native USDC tokens, and Circle TokenMessengerV2. A generic Forge build had different metadata because of remappings; the verified build reproduced the chain code. These comparisons establish deployed code identity rather than relying on an explorer label.

We compared all eight direct creation inputs, factory-call arguments, constructor values, and on-chain getters. The four Arbitrum internal factory-child init-code blobs also matched the verified build. Base internal init-code blobs were not available from the accessible explorer/RPC services; exact factory runtime, call inputs, CREATE-derived addresses, child runtimes, constructor events, and bindings independently corroborate the Base result. These checks do not establish the deployer's exact workstation commit or command; the merged Base deployment-repository pin was recorded after on-chain deployment, although a predeployment commit contained the same Base scripts, inputs, build settings, and direct source pins.

On Arbitrum, the Controller binds to existing proxy [0x92afd6F](#) and the facet binds to native USDC [0xaf88d06](#). On Base, those addresses are proxy [0x2917956](#) and native USDC [0x833589f](#). Both CCTP facets bind to Circle TokenMessengerV2 [0x28b5a0e](#). Each PAUFactory points to its chain's registered Beacon; each Controller points to that Beacon, its registered AccessControls and RateLimits, and the existing proxy. Explorer source information supported these comparisons, but independently rebuilt code and chain state determined the result.

Integration and selector configuration

We enumerated the integration entries on each Beacon and Controller, then read every selector mapping. Each Beacon contains exactly one `CCTP_FACET` entry pointing to the registered facet. Each Controller has copied that same integration and contains exactly ten unique CCTP selector pairs. We checked the facet's delegate selectors, the eleven built-in Controller selectors for collisions, and the lack of routes for direct proxy calls or delegatecalls through the Controller. Short or unwired fallback calls revert. The Controller has no public initializer or upgrade entrypoint; its dispatch and CCTP domain storage use distinct namespaces. These checks test whether an outsider or relayer can steer delegatecall or bypass the transfer and domain-setter roles. No such bypass was found in the present one-facet map.



The ten selector pairs match the CCTP integration on the Sky PAU Beacon on Ethereum as a set; different ordering and chain-specific facet addresses are expected. Checking both Beacon and Controller is necessary because synchronization is explicit. A test-only malicious facet showed that, after a Beacon administrator registers new wiring, an AccessControls administrator syncs it, and governance grants the Controller proxy authority, even an outsider could reach an unguarded facet and use arbitrary proxy calls, including `doDelegateCall`. That facet is absent from both deployments. The test establishes the trust boundary for future facets, not a current actor-accessible exploit.

Roles, Safe and existing proxy

We checked the complete member sets of enumerable new-contract roles and reconstructed non-enumerable RateLimits roles from their full event history, then compared them with direct permission queries.

On both chains, `SPARK_EXECUTOR` was the sole Beacon, AccessControls, and RateLimits administrator and the sole Agent administrator. The Agent alone held `ALLOCATOR_ROLE`; the PAU Controller alone held `CONTROLLER` on PAU RateLimits. The primary relayer was the Agent's sole actor, the grantor Safe its sole grantor, and the freezer Safe its sole revoker. The deployer no longer held these permissions after configuration. Counts, addresses, and the grant/revoke sequence matter because checking only an intended holder would miss an extra privileged holder.

The observed grantor Safe owner list has five members and a 3-of-5 threshold on both chains. The relayer Safe has a 1-of-2 threshold and the freezer Safe a 2-of-4 threshold. We checked those observed owner lists and thresholds, not independent organizational approval of their signer policies. Safe implementation version, modules, guard, and fallback handler were not established by this review and are not relied on for the conclusion.

The existing `ALM_PROXY` retained its code and PR-relevant role values during deployment and configuration. The legacy ForeignController remained its authorized `CONTROLLER`; the new PAU Controller had not received that role. We checked relevant proxy holders directly and scanned proxy/legacy role events through the current snapshots. The proxy's roles are not enumerable, so this is not a claim to reconstruct every historical proxy role. These comparisons address the risk that registering a new Controller changed custody authority.

Inactive state and functional checks

At the post-configuration and complete later state blocks, both PAU CCTP rate-limit records were zero, Ethereum domain 0 had no PAU recipient or fee bounds, and the new Controller lacked proxy authority. The planned Spark hot-wallet actor, Soter grantor and revoker, and PAS admin grants on both `PAU_ACCESS_CONTROLS` and `PAU_RATELIMITS` were also absent. The new Controller still lacked proxy `CONTROLLER` at the later factory/role follow-up blocks, and no new-stack configuration or role event followed the final configuration transaction through those samples. A relay transfer through the Agent passed the allocator check but reverted at the zero rate limit. Direct relay calls, unauthorized Agent calls, unmapped selectors, direct facet calls, and direct proxy delegatecalls also failed their respective checks. We found no production PAU CCTP burn through the review anchors. These independent barriers explain why the new route is inactive; they are not substitutes for verifying the future spell.

Across the two chains, 16 production post-deployment state tests, 52 local/live E2E tests, five Arbitrum event tests, 20 focused dispatch/authorization fork tests, and 12 factory-insertion fork tests passed. An independent local factory-insertion proof also passed. The E2E and focused checks covered simulated activation, CCTP burn and relay, allowance cleanup, role separation, rate limits, integration removal, the Agent self-renunciation path, and factory nonce preemption.

Before the route is treated as active, the executed spell should be checked for the exact proxy-role grantee and receipt-derived Controller bindings, Ethereum recipient, fee bounds, both finite PAU rate limits, the relay Safe and Spark hot-wallet actors, the Soter grantor and revoker, PAS admin on both `PAU_ACCESS_CONTROLS` and `PAU_RATELIMITS`, and the scheduled Beacon-admin handover. Its limits should be assessed together with the still-active legacy limits, since both routes draw from the same proxy. Any later facet addition or change in the permission graph calls for a new review of the shared-proxy behavior. These are boundaries of this deployment conclusion, rather than defects in the observed pre-activation state.

Shared-proxy architecture and approval limits

On both chains, proposed activation would give two Controllers unrestricted proxy authority through `CONTROLLER`, while their own caller rules and rate limits remain separate. The same 1-of-2 relay Safe can reach both CCTP routes. At the reviewed snapshots, Arbitrum's legacy domain-0 limit was 50 million USDC with about 25 million USDC per day of refill; Base's was 200 million USDC with about 500 million USDC per day of refill. The legacy aggregate CCTP key was



unlimited. New PAU limits were zero. Once enabled, new and legacy budgets add rather than enforcing a custody-wide cap. The 10 million maximum and 50 million/day refill in E2E tests are examples, not actual spell values. Governance must check finite new limits and the combined capacity before activation.

The current PAU value-moving path has only the CCTP facet wired, with fixed USDC and Circle targets and no caller-selected external target. The Controller fallback has no global caller check, but `transfer` requires `ALLOCATOR_ROLE` and `setDomainParameters` requires `DEFAULT_ADMIN_ROLE`. Focused probes and fork tests found no actor or outsider path to choose a different facet, bypass those roles, or invoke arbitrary proxy `doDelegateCall` under this wiring.

At the reviewed blocks, the Executor could change Beacon wiring and sync the Controller, and had zero local delay; upstream governance controls were the relevant administrative boundary. After the planned handover, Sky's `L2GovernanceRelay` controls Beacon configuration, while the Executor retains AccessControls admin authority to sync it into the Controller or remove Controller integrations; PAS may receive that authority too. The handover does not change the pre-activation approval or conditional CCTP-only conclusion for the reviewed code and state. A test-only malicious facet showed why any authorized future facet or wire change needs its own review. The current one-facet result cannot be generalized to all future facets.

Each Controller's reentrancy guard protects its own storage, not the shared proxy. We traced the previously raised cross-controller balance-delta concern: the relevant legacy before/after proxy balance check is for an aToken in `depositAave`, whereas the new CCTP facet burns USDC and has no inflow or balance-delta calculation. The observed fixed USDC and Circle endpoints are not current Agent actors or legacy relayers. We found no concrete cross-controller accounting exploit in this CCTP-only combination. A new facet, privileged callback target, or changed legacy/new path could alter that conclusion.

Incident response must address the two routes separately: removing an Agent actor does not remove its legacy `RELAYER` role, and an authorized grantor can later add an Agent actor. Revoking the new Controller's proxy role is the clearest custody stop for that path but requires proxy admin authority; a completed CCTP burn cannot be reversed locally. The Agent actor can also cause the Agent to renounce its own `ALLOCATOR_ROLE`, disabling new-route transfers until the Executor restores the role. That is a recoverable availability issue, not a recipient change or fund-drain path. The architecture is conditionally acceptable for the exact CCTP-only code, wiring, and roles reviewed; actual activation remains subject to spell and post-execution checks.



Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	-	-	-
Informational	1	1	-
Total	1	1	-

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				



Detailed Findings

ID	Title	Severity	Status
I-01	An attacker could have claimed the expected Controller address during deployment	Informational	Acknowledged

Informational Severity Issues

I-01: An attacker could have claimed the expected Controller address during deployment

Files:

spark-pau-deploy/blob/master/script/parallel-controller/O-DeploySparkPAUParallel.s.sol

Description:

The [PAUFactory](#) lets anyone deploy its child contracts. Each deployment uses **CREATE**, so the child address depends on the factory's nonce. The [deployment script](#) broadcasts separate factory calls and exports the addresses returned during script simulation. It does not check those addresses against the executed transactions. An attacker's call executed between the deployer's calls could therefore change the address of a later child.

After the deployer creates AccessControls at factory nonce 1 and RateLimits at nonce 2, an attacker could call **deployController** before the deployer's Controller call. The attacker's Controller would occupy the predicted nonce-3 address and point to AccessControls and RateLimits that the attacker administers. The attacker could grant the deployer co-admin rights so configuration succeeds while the attacker retains admin rights. The deployer's Controller call would then create a different Controller at nonce 4.

The [configuration script](#) takes a Controller address from a static input and trusts its AccessControls and RateLimits getters. It does not check the address against the executed deployment results or check for an extra admin after handoff. If the input holds the stale nonce-3 address, the script can configure the attacker's Controller and revoke the deployer's admin rights while leaving the attacker's rights intact.

The substituted Controller cannot move proxy assets until it receives a proxy role. If governance later grants **CONTROLLER** on the existing **ALMProxy** to the nonce-3 address, the attacker can grant itself **ALLOCATOR_ROLE**, set the Ethereum-domain CCTP recipient to its own address, and call **cctp_transfer**. The attacker can also set the aggregate and domain rate limits to unlimited. Repeated burns through the genuine, fixed CCTP V2 facet could consume the proxy's available



USDC and later inflows while that authority remains. Circle's per-message burn limit and transaction gas limit individual calls, but do not cap the cumulative amount.

This did not occur in PR 115's deployments. The Base and Arbitrum factory transactions, child CREATE nonces, Controller bindings, and role histories match the intended deployment. Both PAUFactories ended at nonce 4 and both AgentFactories at nonce 2, with no extra child creation or retained attacker admin.

Recommendations:

For future deployments, deploy the children atomically or restrict who can call the factory. Before configuration or a proxy-role grant, verify the Controller address against the executed deployment receipt, check its dependency addresses, and confirm that no unexpected admin remains.

Customer Response:

Acknowledged. For PR #115 the deployment was checked after the fact: factory nonces, CREATE-derived addresses, receipts and role histories match, as Certora confirms. The post-deploy tests in spark-pau-deploy pin exact event counts per contract. Before the activation spell grants **CONTROLLER** on **ALM_PROXY**, the spell tests assert the grantee is **PAU_CONTROLLER** from the registry, which is the verified nonce-3 child. For future deployments we will assert factory provenance (the controller's address equals the address the executed receipt returned, and its dependencies are the deployed AccessControls and RateLimits with no extra admin) before any role handoff.



Conclusions

We completed the deployment verifications required by [PR #115](#) on Arbitrum and Base. We matched each registered address to its deployment transaction, factory nonce, code, and constructor values. We checked the CCTP selectors, roles, configuration history, and existing [ALM_PROXY](#). We also reviewed the deployment scripts and tests, investigated the factory frontrunning issue, and reviewed the design with two Controllers from first principles. We found no deployment mismatch, attacker-controlled contract, or unexpected proxy authority on either chain.

We approve the Arbitrum One deployment. We approve the Base deployment. With the code and roles we reviewed, we found no way for a relayer or outsider to steal proxy funds or run arbitrary code. The old and new routes have separate rate limits and need separate emergency actions. The factory race can threaten future deployments, but it did not affect these deployments.

Before activation, the actual governance spell and its executed state must be reviewed. The proxy role, Ethereum recipient, fee bounds, rate limits, role grants, and unchanged CCTP wiring must be confirmed. The combined capacity of the old and new routes must also be assessed.



Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.